

Organización y gestión de la memoria

Diego Feroldi*

Erica Vidal

Arquitectura del Computador

Departamento de Ciencias de la Computación

FCEIA-UNR



* Actualizado 22/04/2026 (D. Feroldi, feroldi@fceia.unr.edu.ar)

Índice

1. Introducción	1
2. Memoria virtual	1
3. Espacio de direcciones virtuales	4
4. Asignación contigua de memoria	7
5. Segmentación	9
6. Paginación	11
7. La tabla de páginas	12
8. Tamaño de página	15
9. Tablas de páginas multinivel	16
10. Translation Lookaside Buffer (TLB)	18
11. Protección de memoria	20
12. Políticas de reemplazo	20
13. Caso de estudio: Intel Core i7	21
A. Unidades de memoria	23

Nota general

Este apunte no es para nada una referencia completa del tema **Organización y gestión de memoria** sino que debe ser utilizado como material complementario a lo visto en las clases teóricas.

1. Introducción

La memoria es uno de los componentes centrales de un sistema de cómputo y su organización influye de manera directa en el rendimiento, la eficiencia y la seguridad del sistema operativo. Los programas y los datos que estos utilizan deben residir en memoria para poder ser ejecutados, por lo que resulta esencial contar con mecanismos adecuados que permitan administrar este recurso limitado de forma correcta y eficiente.

Este apunte aborda los principios fundamentales de la **organización y gestión de la memoria**, analizando cómo el sistema operativo asigna, protege y traduce las direcciones de memoria utilizadas por los procesos. Se estudian los distintos esquemas de asignación de memoria, el concepto de direcciones lógicas y físicas, los mecanismos de vinculación de direcciones (*address binding*) y el rol del hardware de soporte, en particular la unidad de gestión de memoria (MMU).

Observación

Dentro de la jerarquía de memoria, existen dos capas cruciales: la memoria principal y la memoria secundaria.

- La **memoria principal**, también llamada **memoria primaria**, es una memoria utilizada para almacenar programas mientras se ejecutan. Típicamente consiste en DRAM en las computadoras actuales.
- La **memoria secundaria** es una memoria no volátil utilizada para almacenar programas y datos con mayor capacidad que la memoria principal. Normalmente se utilizan discos magnéticos pero hay otros ejemplos tales como discos de estado sólido, cintas magnéticas, CDs, DVDs, memorias USB, etc.

2. Memoria virtual

La memoria consiste en un gran arreglo de bytes, cada uno con su propia dirección. La CPU obtiene las instrucciones desde la memoria de acuerdo con el valor del contador de programa. Estas instrucciones pueden provocar cargas adicionales desde direcciones de memoria específicas y almacenamientos en ellas. Para administrar la memoria de manera más eficiente y con menos errores, los sistemas modernos proporcionan una abstracción de la memoria principal conocida como **memoria virtual**.

Las primeras computadoras contaban con muy poca memoria física. Uno de los primeros sistemas de tiempo compartido trabajaba en una computadora PDP-1 con un tamaño de memoria principal de hasta 144 KB para almacenar el sistema operativo y los programas de usuario. Si un programa necesitaba más espacio de memoria había que usar memoria secundaria, pero era el programador el que debía dividir el programa en varios fragmentos que entraran en memoria. Luego, para ejecutar su programa, debía hacer que se lea el primer fragmento, se ejecutara un

tiempo y al terminar se cargara el siguiente fragmento y así sucesivamente sin ayuda del sistema. En 1961 se propuso un método para realizar automáticamente la gestión entre la memoria principal y secundaria. Este método que ahora se conoce como **memoria virtual** se usó por primera vez en los años 70 en varias computadoras.

A través del mecanismo de memoria virtual, cada proceso accede a su propio **espacio de direcciones virtuales** privado (también conocido simplemente como **espacio de direcciones**), que el sistema operativo mapea de algún modo en la memoria física de la máquina. Una referencia de memoria dentro de un programa en ejecución no afecta el espacio de direcciones de otros procesos (ni del propio sistema operativo); desde la perspectiva del programa en ejecución, parece tener toda la memoria física para sí mismo. Sin embargo, la realidad es que la memoria física es un recurso compartido, gestionado por el sistema operativo.

La memoria virtual es un mecanismo eficiente que ofrece a cada proceso un espacio de direcciones amplio, uniforme y privado. Este mecanismo proporciona tres capacidades clave:

1. Utiliza la memoria principal de manera eficiente manteniendo solo las áreas activas en la memoria principal y transfiriendo datos entre el disco y la memoria según sea necesario.
2. Simplifica la gestión de la memoria al proporcionar a cada proceso un espacio de direcciones uniforme.
3. Protege el espacio de direcciones.

En base a lo anterior, la memoria virtual se puede definir como una técnica que utiliza la memoria principal como un “buffer” de almacenamiento para el almacenamiento secundario. La idea consiste en separar los conceptos de espacio de direcciones y posiciones de memoria física, de modo que las direcciones utilizadas por los programas no coincidan necesariamente con las ubicaciones reales en la memoria principal.

Ejemplo

Consideremos una computadora de los años 60 con una memoria de 4 KB y palabras de 16 bits. Un programa para esta computadora puede direccionar 65536 (2^{16}) palabras de memoria. Eso es porque existen 65536 direcciones de 16 bits, cada una de las cuales corresponde a una palabra de memoria diferente.

Entonces, el espacio de direcciones virtuales de esta computadora corresponde con el conjunto de números 0, 1, 2, ..., 65535, porque ese es el conjunto posible de direcciones virtuales. Por otro lado, tenemos las direcciones de memoria física cuya cantidad depende de la memoria física de la máquina. Si contamos con 4 KB de memoria RAM, podemos almacenar 4096 palabras de 16 bits, utilizando direcciones de memoria físicas que van desde 0 hasta 4095.

Observación

*El número de **direcciones virtuales** depende únicamente del número de bits que tiene una dirección y es independiente de la cantidad de memoria física que tiene la máquina.*

Antes de la introducción de la memoria virtual, los programas utilizaban direcciones físicas reales, es decir, las direcciones generadas por el CPU correspondían directamente a posiciones en la memoria principal. Por ese motivo, no existía una separación conceptual entre espacio de direcciones del proceso y memoria física. La idea de separar el espacio de direcciones y las direcciones de memoria es la siguiente. En cualquier momento se puede acceder a una posición de memoria, pero no es necesario utilizar directamente la dirección de memoria física.

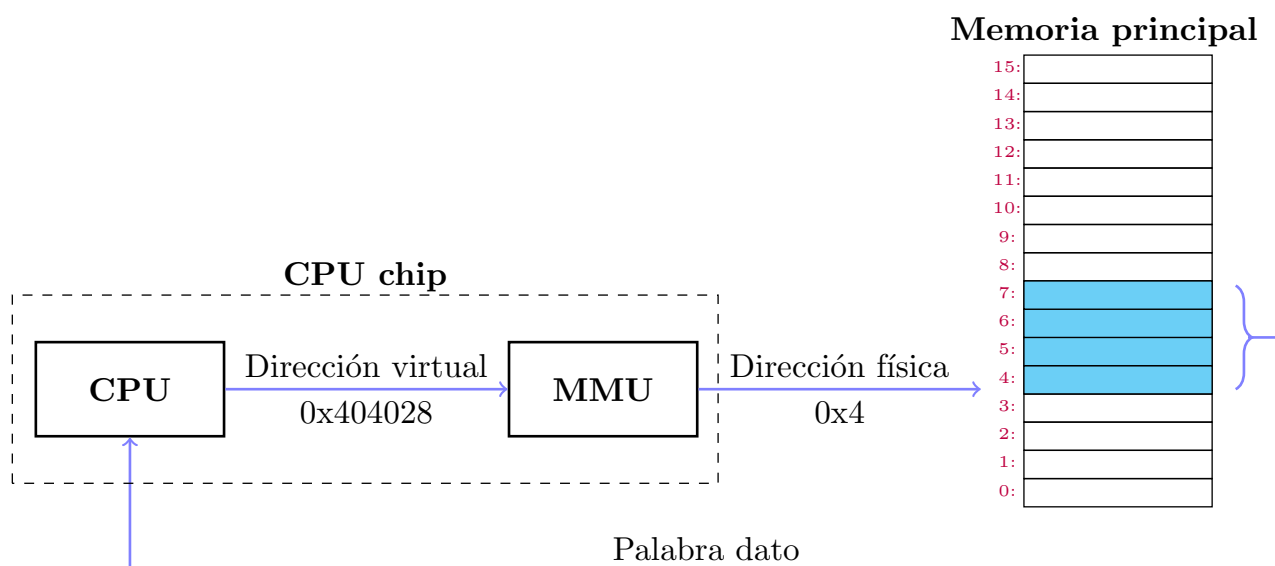


Figura 1: Esquema básico de memoria virtual.

Se utiliza una dirección de memoria virtual que tiene una correspondencia con una dirección de memoria física. En otras palabras, se define un “mapeo” del espacio de direcciones virtuales a las direcciones de memoria reales (físicas).

Observaciones

- La motivación principal para utilizar memoria virtual es permitir el uso compartido eficiente y seguro de la memoria entre varios programas.
- La segunda motivación es permitir que un único programa de usuario, o incluso varios de manera simultánea, excedan el tamaño de la memoria primaria. Estas cuestiones se desarrollan con mayor detalle a lo largo de este apunte.

En la Fig. 1 se muestra un ejemplo de mapeo donde la dirección lógica (o virtual) 0x404028 se corresponde con la dirección física 0x4 de la memoria principal. Todas las implementaciones de memoria virtual —excepto en emuladores— requieren soporte en hardware. La traducción en tiempo de ejecución de direcciones virtuales a direcciones físicas la realiza un dispositivo de hardware llamado **unidad de gestión de memoria** (*memory-management unit*, MMU).

Observación

Una **dirección lógica** es una dirección de memoria tal como la ve una aplicación. Por ejemplo, en la instrucción `movl 0x404028, %eax` hay una dirección lógica: 0x404028. Un programador tiene la ilusión de que él es el único usuario de la memoria. Cualquiera que sea la celda de memoria a la que se dirija, nunca ve datos o instrucciones de otros programas que se ejecutan con el suyo en paralelo. Sin embargo, la memoria física contiene varios programas a la vez. De hecho, otro programa podría ejecutar la misma instrucción pero no accedería a las mismas celdas en memoria. Esto es debido al mecanismo de memoria virtual que iremos viendo a continuación. En este contexto, **dirección virtual** es sinónimo de **dirección lógica**.

3. Espacio de direcciones virtuales

El **espacio de direcciones virtual** es una abstracción de la memoria física que representa la visión del programa en ejecución sobre la memoria del sistema. Comprender esta abstracción fundamental del sistema operativo es esencial para entender cómo se virtualiza la memoria.

El espacio de direcciones de un proceso incluye todo el estado de memoria del programa en ejecución. Por ejemplo, el código del programa (las instrucciones) debe residir en algún lugar de la memoria, por lo que forma parte del espacio de direcciones. Mientras se ejecuta, el programa utiliza una pila (*stack*) para registrar su posición en la cadena de llamadas a funciones, asignar variables locales, pasar parámetros y devolver valores a las rutinas. Por último, el *heap* se emplea para la memoria gestionada dinámicamente por el usuario, como la que se asigna mediante `malloc()` en C o `new` en lenguajes orientados a objetos como C++ o Java. Además, el espacio de direcciones puede contener otros elementos, como variables inicializadas de forma estática.

El modelo de espacio de direcciones virtuales se puede dividir en regiones o segmentos. Un **segmento** es una porción contigua del espacio de direcciones de una longitud determinada. Los segmentos son una forma de organizar y administrar el espacio de direcciones de la memoria virtual. Representan divisiones lógicas dentro de ese espacio de direcciones y cada segmento puede tener sus propios permisos, tamaño y características. Así, los segmentos se definen como bloques de datos de diferentes longitudes que residen en la memoria secundaria. Un segmento completo puede copiarse temporalmente en una región disponible de la memoria principal. Cada segmento consiste en una sucesión lineal de direcciones, de la 0 a alguna dirección máxima y por lo tanto la longitud de cada segmento puede ser cualquiera desde 0 hasta el máximo permitido. Diferentes segmentos pueden tener diferentes longitudes, y generalmente así es. Además, la longitud de los segmentos podría cambiar durante la ejecución. Por ejemplo, la longitud de un segmento de pila podría incrementarse cada vez que algo se mete en la pila y reducirse cada vez que algo se “desapila”.

El modelo de espacio de direcciones virtuales puede dividirse en regiones o segmentos. Un **segmento** es una porción contigua del espacio de direcciones, de longitud variable, que permite organizar y administrar la memoria virtual de acuerdo con la estructura lógica de los programas. Cada segmento representa una división lógica dentro del espacio de direcciones y puede tener permisos, tamaño y características propios. Los segmentos se conciben como bloques de información de distintas longitudes que residen en la memoria secundaria y que pueden copiarse temporalmente en regiones disponibles de la memoria principal. Cada segmento consiste en una sucesión lineal de direcciones, desde la dirección 0 hasta una dirección máxima, por lo que su longitud puede variar desde 0 hasta el máximo permitido. En general, distintos segmentos tienen longitudes diferentes y estas pueden modificarse durante la ejecución del programa. Por ejemplo, la longitud de un segmento de pila puede incrementarse cada vez que se incorpora un nuevo elemento y reducirse cuando dicho elemento se retira.

El espacio de direcciones virtuales de un proceso en Linux se puede dividir en 5 segmentos:

- **Segmento de texto:** En este segmento van todas las instrucciones. El segmento de texto se inicializa mediante la directiva `.text` en el GNU assembler (GAS).
- **Segmento de datos inicializados:** Este segmento contiene todos los datos estáticos inicializados al iniciar el programa. Este segmento se inicializa mediante la directiva `.data`.
- **Segmento de datos no inicializados:** Este segmento contiene todos los datos no inicializados al iniciar el programa y se utiliza para reservar espacio de memoria para variables no inicializadas. Este segmento se inicializa mediante la directiva `.bss`. El término “bss” proviene de “*Block Started by Symbol*”.

- **Segmento heap:** En este segmento se almacenan los datos asignados mediante `malloc` o `new`. A diferencia de los segmentos anteriores, el heap puede crecer de manera dinámica, por ejemplo, al realizar una llamada a `malloc`.
- **Segmento pila:** Este segmento es la pila en tiempo de ejecución. En este segmento se encuentran los siguientes elementos:
 - direcciones de retorno
 - algunos parámetros de la función
 - variables locales de funciones
 - espacio para variables temporales

El segmento de pila también crece de manera dinámica.

La Fig. 2 muestra el modelo de espacio de direcciones virtuales de un proceso en Linux. El código del programa se encuentra en la parte inferior del espacio de direcciones¹. Dado que el código es estático (y, por lo tanto, fácil de ubicar en memoria), podemos colocarlo en la parte inferior del espacio de direcciones y estar seguros de que no necesitará más espacio durante la ejecución del programa. De manera similar, a continuación se puede ubicar el segmento con las variables inicializadas y el segmento con las variables no inicializadas, que no crecerán durante la ejecución.

A continuación, tenemos las dos regiones del espacio de direcciones que pueden crecer (y reducirse) durante la ejecución del programa. Estas son el *heap* (ubicado después del segmento de datos inicializados) y la pila (en la parte superior). Se colocan de esta manera porque ambas requieren la capacidad de crecer, y al ubicarlas en extremos opuestos del espacio de direcciones, se facilita ese crecimiento: cada una puede expandirse en direcciones opuestas. Así, el *heap* comienza justo después de los datos no inicializados y crece hacia arriba (por ejemplo, cuando un usuario solicita más memoria mediante `malloc()`), mientras que la pila está ubicada en la parte superior y crece hacia abajo (por ejemplo, cuando un usuario realiza una llamada a procedimiento). Sin embargo, esta disposición de la pila y el *heap* es solo una convención; el espacio de direcciones podría organizarse de manera diferente.

La pila se asigna a las direcciones más altas de un proceso y en Linux x86-64 es `0x7fffffffffff` o 131 TB². Esta dirección es equivalente a 47 bits con todos los bits en 1. Tanto el *heap* como la pila necesitan crecer mientras el proceso está en ejecución: el *heap* crece “hacia arriba” (hacia direcciones de memoria mayores), mientras que la pila crece “hacia abajo” (hacia direcciones de memoria menores). Ambos segmentos pueden llegar a encontrarse en el medio y, en consecuencia, provocar una colisión. El uso del espacio de *heap* y de pila en lenguaje ensamblador no implica la existencia de segmentos con esos nombres específicos, sino el manejo explícito de direcciones de memoria.

El segmento de pila está limitado por el kernel de Linux. El tamaño típico es de 16 MB para Linux de 64 bits. Esto se puede inspeccionar usando “`ulimit -a`”. 16 MB parece bastante pequeño, pero está bien a menos que se usen matrices grandes como variables locales en las funciones. El rango de direcciones de la pila es `0x7fffff000000` a `0x7fffffffffff`. El kernel reconoce si ocurre una falla en las direcciones fuera de este rango (*segmentation fault*).

El segmento de datos inicializados contiene las variables globales y estáticas del programa que han sido inicializadas con valores distintos de cero. El comienzo de este segmento se indica

¹En realidad, por debajo del segmento de código se encuentra memoria virtual no asignada, es decir, direcciones que no están mapeadas a ningún segmento válido del proceso.

²Esto es realidad no es exactamente así. La región superior del espacio de direcciones está reservada para el núcleo (Memoria virtual del núcleo) pero para los fines prácticos podemos asumir que la región superior es la pila.

Espacio de direcciones virtuales

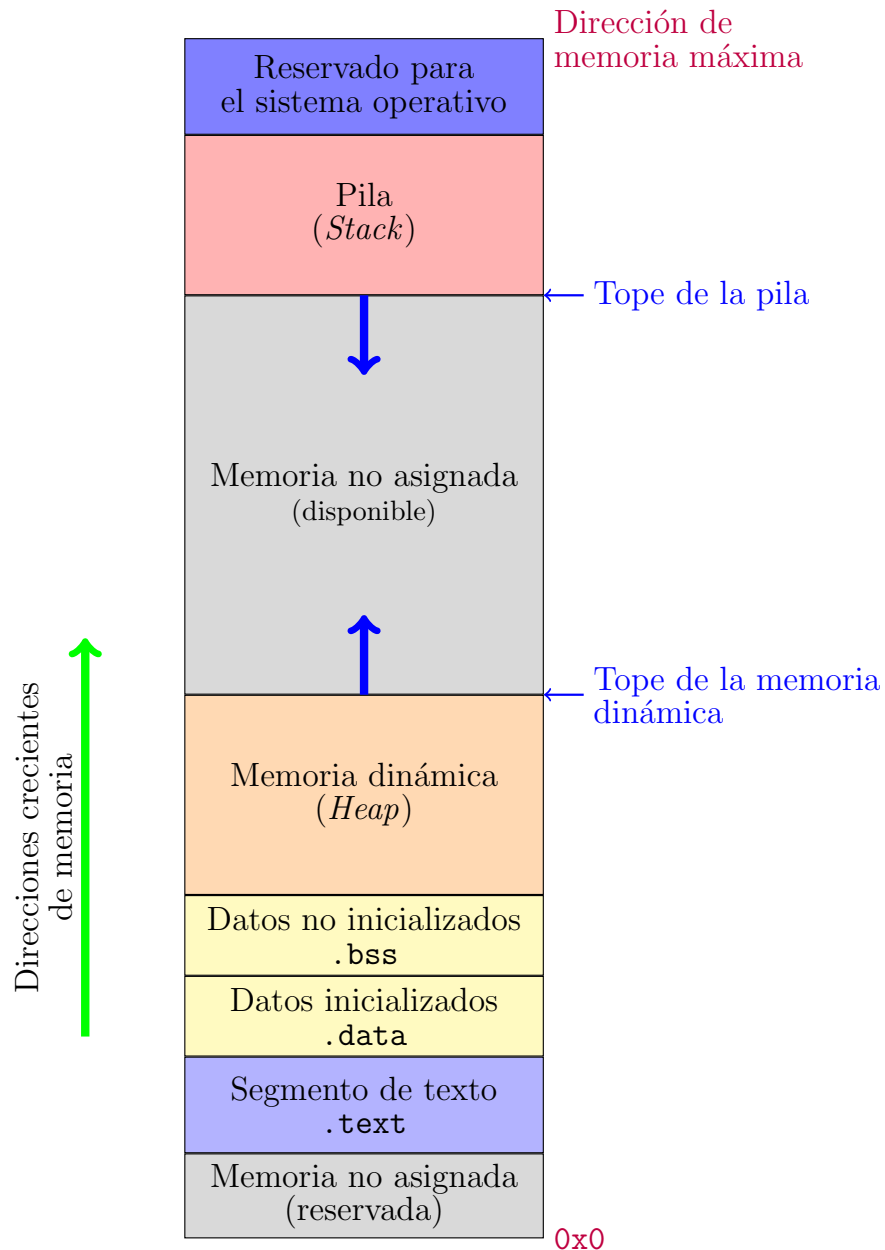


Figura 2: Modelo de memoria de un proceso en Linux.

mediante la directiva `.data`. Este segmento puede incluir regiones de solo lectura y de lectura-escritura. En consecuencia, en este segmento se almacenan variables como las siguientes:

```
char cadena[] = "Hola Mundo";    //global
int contador = 1;                 //global
const int num = 5;               //global
```

Las primeras dos se almacenan como datos de lectura-escritura, mientras que la última lo hace como sólo lectura.

El segmento de datos no inicializados almacena todas las variables globales y estáticas que no están inicializadas a cero o no tienen un valor de inicialización en el código fuente. El comienzo de este segmento se indica mediante la directiva `.bss`. Algunos ejemplos son los siguientes:

```
int num;                          //global
static int x=0;
```


Aunque `static int x = 0;` aparece inicializada, el compilador la ubica en `.bss` porque su valor inicial es cero y no requiere espacio en el archivo ejecutable.

El segmento de texto almacena las instrucciones ejecutables del programa, por lo que también se lo denomina segmento de código. Contiene la representación en código de máquina de dichas instrucciones. En general, este segmento puede ser compartido entre distintos procesos, ya que no se modifica durante la ejecución; en particular, cuando incluye código de bibliotecas compartidas, no es necesario duplicarlo en memoria. Por este motivo, suele configurarse como un segmento de solo lectura, con el fin de evitar que un proceso modifique accidentalmente el código del programa.

4. Asignación contigua de memoria

La memoria principal debe alojar tanto al sistema operativo como a los distintos procesos de usuario, por lo que resulta necesario asignarla de la manera más eficiente posible. Uno de los primeros métodos utilizados fue la **asignación contigua de memoria**, en la cual cada proceso se aloja en una única sección de memoria contigua a la sección que contiene al proceso siguiente, como se muestra en la Fig. 3.

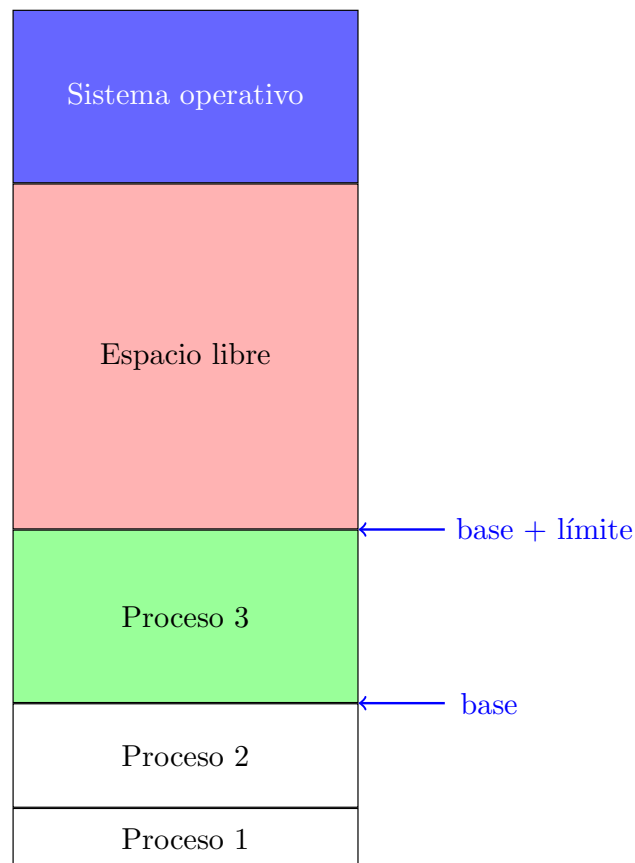


Figura 3: Ejemplo simple de asignación contigua de memoria con tres procesos, donde el proceso 3 está en ejecución.

En este esquema de particiones variables, el sistema operativo mantiene una tabla que indica qué partes de la memoria están disponibles y cuáles se encuentran ocupadas. Inicialmente, toda la memoria está disponible para los procesos de usuario y se considera como un único bloque grande de memoria libre. En este esquema, cada proceso dispone de dos registros especiales:

- un registro base, que indica la dirección física inicial del espacio de memoria asignado al proceso, y
- un registro límite, que define el tamaño máximo del espacio de direcciones del proceso.

Las direcciones generadas por el programa se interpretan como desplazamientos respecto de la base. El hardware suma automáticamente el valor del registro base a la dirección lógica y, además, verifica que el desplazamiento no exceda el valor del registro límite. Si la verificación falla, se genera una excepción de protección. La Fig. 4 ilustra este procedimiento.

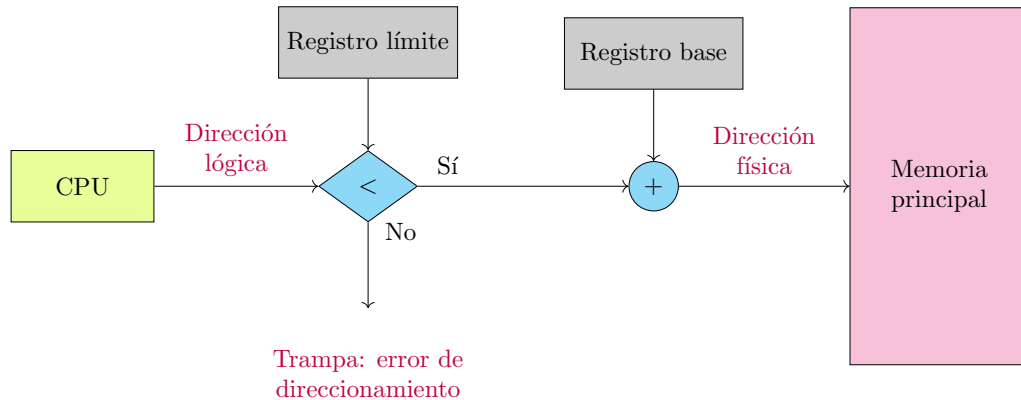


Figura 4: Procedimiento de relocación basado en memoria contigua.

Este método es conceptualmente simple y permite un aislamiento básico entre procesos, pero no admite espacios de direcciones mayores que la memoria física disponible. Otro problema importante es la **fragmentación externa**, que existe cuando hay suficiente espacio total de memoria para satisfacer una solicitud, pero los espacios disponibles no son contiguos. En consecuencia, la memoria queda fragmentada en una gran cantidad de pequeños huecos, como se observa en la Fig. 5. Si bien existen 10 KB de memoria libre, no pueden aprovecharse completamente debido a que están divididos en bloques no contiguos; por lo tanto, no es posible crear un nuevo proceso de 5 KB. A continuación se presentan otros métodos que ofrecen mayor flexibilidad.

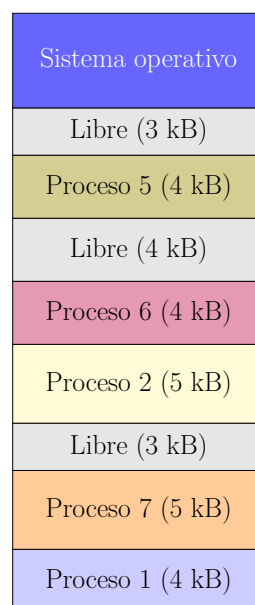


Figura 5: Ejemplo de fragmentación externa.

5. Segmentación

La segmentación es un método de memoria virtual basado en la división del espacio de direcciones en segmentos. Como se mencionó, cada segmento se caracteriza por una dirección base y un límite, que permiten al hardware traducir las direcciones generadas por el programa a posiciones de memoria y, al mismo tiempo, verificar que los accesos se realicen dentro de los límites válidos. De este modo, la traducción de direcciones no solo permite ubicar un segmento en memoria, sino también detectar accesos inválidos y aplicar mecanismos de protección, reforzando el aislamiento y la seguridad entre procesos.

En segmentación una dirección lógica se considera que está formada por dos partes: un número de segmento, que se asocia a una dirección física, y un desplazamiento de segmento. Una consecuencia de que los segmentos sean de tamaño desigual es que no existe una relación simple entre las direcciones lógicas y las direcciones físicas. Un esquema de segmentación simple hace uso de una tabla de segmentos para cada proceso³ e información sobre los bloques libres de memoria principal. Cada entrada de la tabla de segmentos tiene que dar la dirección inicial en la memoria principal del segmento correspondiente. La entrada también debe proporcionar la longitud del segmento, para garantizar que no se utilicen direcciones no válidas. Cuando un proceso ingresa al estado “En ejecución”, la dirección de su tabla de segmentos se carga en un registro especial utilizado por el hardware de administración de memoria⁴.

La Fig. 6 muestra de manera esquemática cómo funciona segmentación. En este ejemplo vemos 5 segmentos, su correspondiente tabla de segmentos y cómo se ubican estos segmentos en la memoria física en base a la información de la tabla.

Espacio lógico de direccionamiento

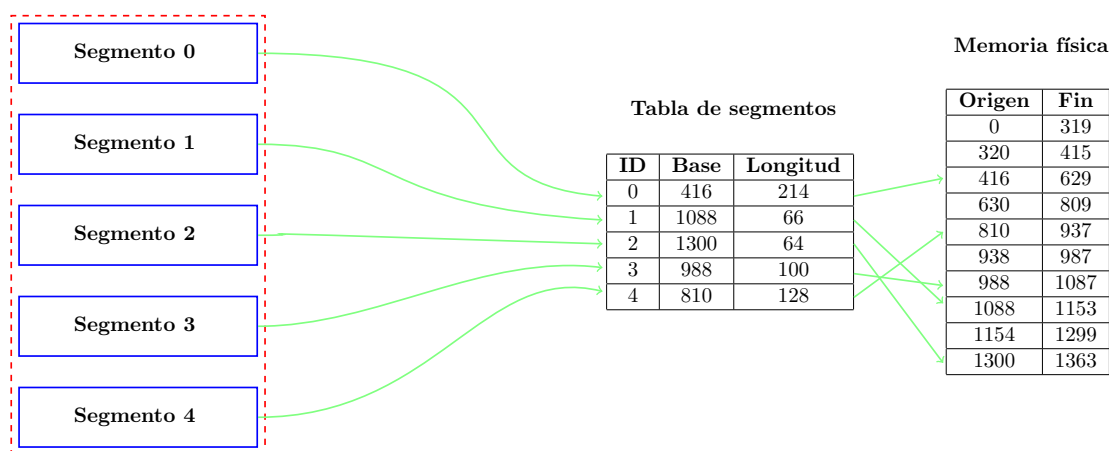


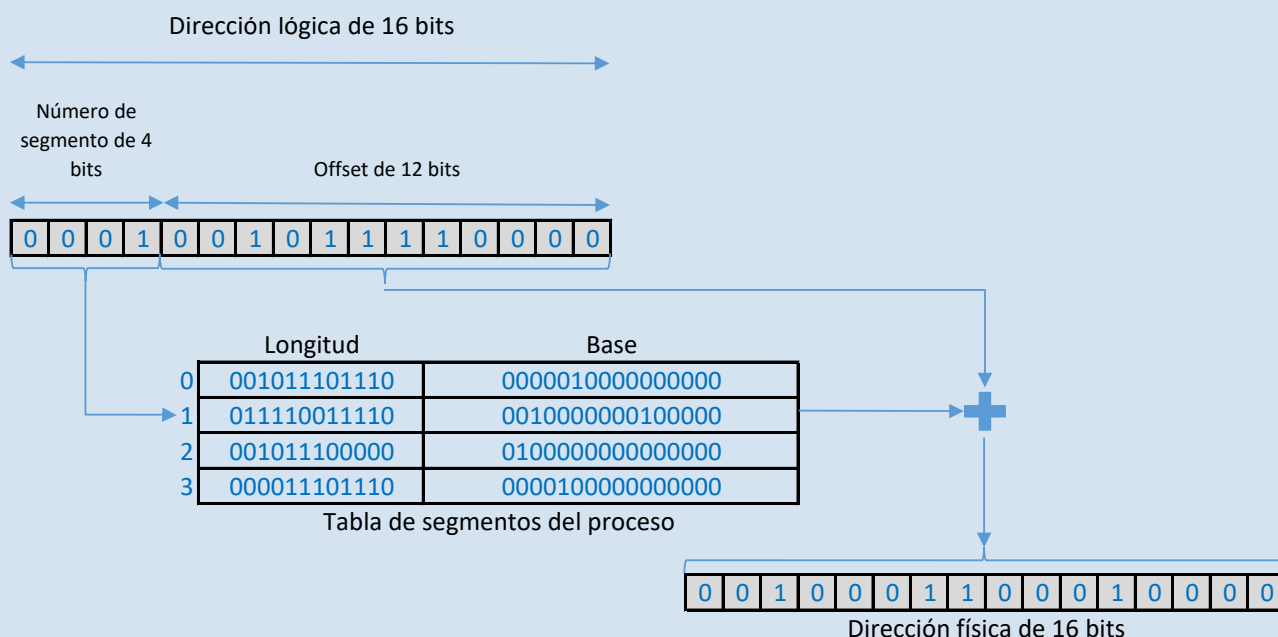
Figura 6: Ejemplo de esquema de segmentación (ID es el identificador del segmento).

Ejemplo

Consideremos una dirección de $n+m$ bits, donde los n bits más a la izquierda son el número de segmento y los m bits más a la derecha son el desplazamiento. En nuestro ejemplo, $n = 4$ y $m = 12$. Por lo tanto, el tamaño máximo del segmento es $2^{12} = 4096$. Este ejemplo se ilustra en la siguiente figura:

³Un proceso, en informática, puede entenderse informalmente como una instancia de un programa en ejecución. Este tema se verá en detalle en Sistemas Operativos II.

⁴Este mecanismo depende de la arquitectura.



Debemos realizar los siguientes pasos para la traducción de la dirección:

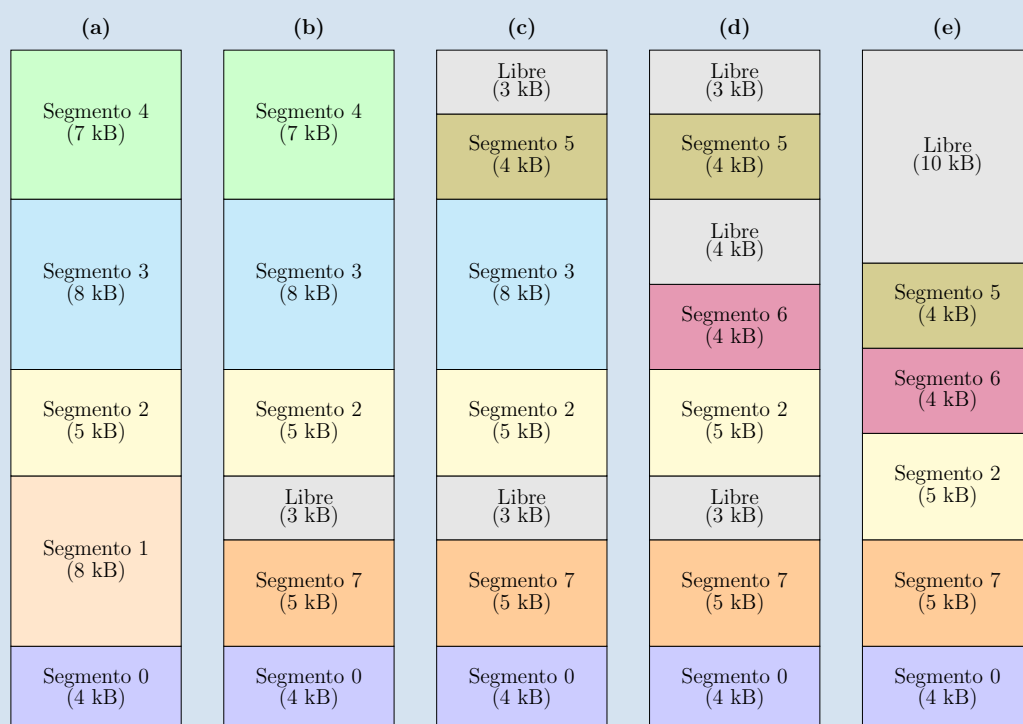
- Extraer el número de segmento como los n bits más a la izquierda de la dirección lógica.
- Utilizar el número de segmento como índice en la tabla de segmentos del proceso para encontrar la dirección física inicial del segmento. En este ejemplo hay 4 segmentos.
- Comparar el desplazamiento, expresado en los m bits más a la derecha, con la longitud del segmento. Si el desplazamiento es mayor o igual que la longitud, la dirección no es válida.
- Finalmente, la dirección física deseada es la suma de la dirección física inicial del segmento más el desplazamiento.

En nuestro ejemplo tenemos la dirección lógica $0x12f0$, que de acuerdo a lo anterior corresponde al segmento número 1 y desplazamiento $0x2f0$. Supongamos que este segmento reside en la memoria principal y comienza en la dirección física $0x2020$. Entonces, la dirección física es $0x2020 + 0x2f0 = 0x2310$.

Observación

En segmentación ocurre un fenómeno denominado **fragmentación externa**, el cual ocurre cuando hay suficiente espacio de memoria total para satisfacer una solicitud, pero los espacios disponibles no son contiguos. Es decir, el almacenamiento queda fragmentado en una gran cantidad de pequeños "huecos". Este problema de fragmentación puede ser grave. Una técnica para superar la fragmentación externa es la **compactación**: de vez en cuando, el sistema operativo mueve los procesos para que sean contiguos y para que toda la memoria libre esté junta en un bloque.

Los siguientes esquemas ilustran este concepto:



Supongamos que inicialmente hay cinco segmentos. Consideremos ahora qué ocurre si el segmento 1 es eliminado y en su lugar se coloca el segmento 7, que ocupa menos espacio. Esto genera la configuración de memoria representada en el esquema **b**. En esta configuración, entre el segmento 7 y el segmento 2 aparece un área desocupada, es decir, un “hueco”.

A continuación, el segmento 4 es reemplazado por el segmento 5, como se muestra en el esquema **e**. Posteriormente, el segmento 3 es sustituido por el segmento 6, generando la configuración que se observa en el esquema **d**.

Consideremos qué ocurriría si quisiéramos agregar un segmento de 8 kB en el esquema **d**. Aunque el espacio total disponible en los huecos es de 10 kB, suficiente en términos absolutos, este espacio está fragmentado en pequeños bloques inútiles, lo que impide cargar directamente el segmento. Por lo tanto, sería necesario eliminar primero algún otro segmento.

Para evitar la fragmentación externa, una posible solución es desplazar los segmentos que están después de cada hueco hacia posiciones más cercanas a la dirección de memoria 0, eliminando así los huecos y dejando un único espacio grande al final. Como alternativa, se puede optar por esperar hasta que la fragmentación externa sea significativa (por ejemplo, cuando un porcentaje considerable de la memoria total esté desperdiciado en huecos) antes de realizar una compactación (o defragmentación).

El esquema **e** ilustra cómo se vería la memoria del esquema **d** tras una compactación. El objetivo de esta operación es consolidar los pequeños huecos en un único bloque grande, permitiendo acomodar uno o más segmentos. Sin embargo, la compactación tiene la desventaja evidente de consumir tiempo para llevarse a cabo. Por lo general, no es práctico compactar cada vez que se genera un hueco, ya que el tiempo invertido sería excesivo.

6. Paginación

En el método de **paginación**, el espacio de direcciones virtuales se divide en **páginas de tamaño fijo**, a diferencia de la **segmentación**, donde se organiza en segmentos de longitud variable. El tamaño de la página es siempre una potencia de 2. El espacio de direcciones físico se

divide de manera análoga en fragmentos del mismo tamaño, denominados **marcos de página física**. En este esquema, una dirección virtual se descompone en un **número de página virtual** y un **desplazamiento**, siendo la cantidad de bits de este último la que determina el **tamaño de la página**.

En la Fig. 7 vemos un esquema de cómo se compone la dirección virtual para un caso con direcciones virtuales de 32 bits y tamaño de página de 4 kB ($4096 \text{ bytes} = 2^{12} \text{ bytes}$). En este caso, el espacio de direcciones virtuales se divide en 2^{20} páginas.

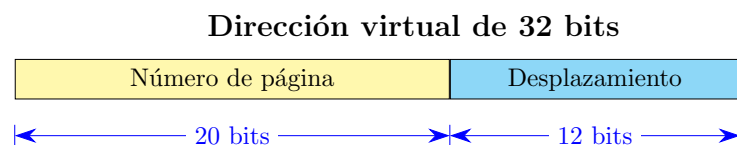


Figura 7: Ejemplo de esquema de dirección virtual utilizando paginación.

El procesador genera una **dirección virtual**, que luego se transforma en una dirección física mediante una combinación de hardware y software. La dirección física es la que realmente se utiliza para acceder a la memoria principal. En la Fig. 8 se muestra un ejemplo de memoria direccionada virtualmente, donde algunas páginas están cargadas en la memoria principal. Este procedimiento se conoce como conversión o **traducción de direcciones**. También puede observarse que no todas las páginas se encuentran en la memoria principal, ya que algunas permanecen en la memoria secundaria.

Observación

En segmentación pura, todos los segmentos de un proceso deben residir completamente en memoria principal. En sistemas avanzados, la segmentación se combina con paginación. En este caso, un segmento puede no estar completamente en memoria principal. Cada segmento se divide internamente en páginas y solo las páginas necesarias del segmento deben estar en RAM. El resto puede residir en memoria secundaria (disco).

7. La tabla de páginas

El procesador utiliza una tabla de páginas para traducir direcciones virtuales en direcciones físicas. Esta tabla de páginas (o tabla de paginación) contiene una entrada por cada página virtual. Cada entrada consiste varios bits destinados a diferentes propósitos (control y estado), además del número de página física representado en binario; entre dichos bits se encuentra el bit de validez (V). Si $V = 1$, indica que la página está en la memoria principal. De lo contrario, la página virtual se encuentra en la memoria secundaria.

Supongamos que la tabla de páginas está organizada como un arreglo contiguo, como se muestra en la Fig. 9. La tabla de páginas está indexada por el número de página virtual (VPN). Por ejemplo, la entrada 3 indica que la página virtual 3 está asignada a la página física $0x7ffe$. En contraste, la entrada 4 no es válida ($V = 0$), lo que significa que la página virtual 4 no se encuentra en la memoria principal, sino en la memoria secundaria.

Para realizar una operación de memoria, el procesador primero debe traducir la dirección virtual en una dirección física y luego acceder a los datos en dicha dirección física. Para ello, consulta la tabla de páginas, que está almacenada en la memoria física, y obtiene el número de página física correspondiente. Si la entrada es válida, combina este número de página física con

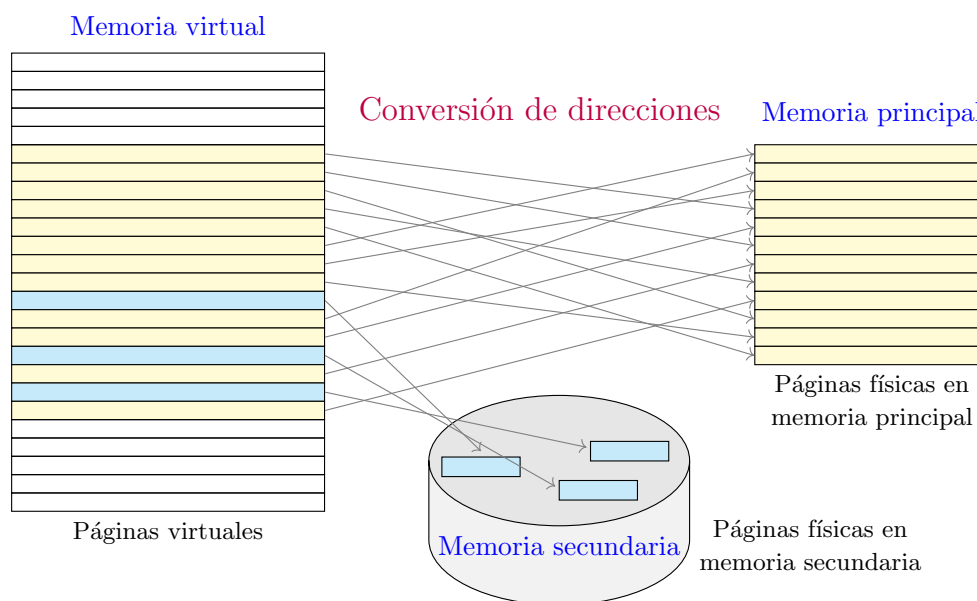


Figura 8: Esquema general de paginación.

el desplazamiento dentro de la página para generar la dirección física. Finalmente, lee o escribe los datos en esa dirección física.

Dado que la tabla de páginas se encuentra en la memoria física debido a su gran tamaño, cada operación de carga o almacenamiento implica, en principio, dos accesos a memoria: uno para leer la tabla de páginas y construir la dirección física, y otro para acceder a los datos propiamente dichos.

Ahora bien, puede ocurrir que la entrada de la tabla de páginas indique que la página solicitada no está cargada en la memoria principal, sino que se encuentra en la memoria secundaria (por ejemplo, en disco). A esta situación se la denomina **fallo de página** (*page fault*). Un fallo de página es una excepción síncrona (o *trap*), precisa y recuperable, generada por hardware que ocurre cuando un proceso intenta acceder a una página virtual que no se encuentra actualmente cargada en memoria principal.

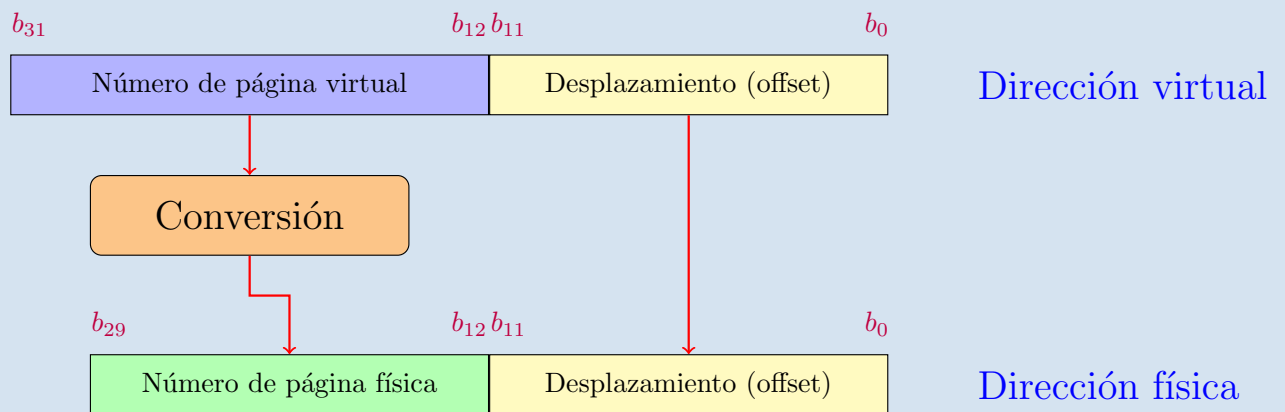
Cuando ocurre un fallo de página, el sistema operativo debe intervenir: detiene la ejecución del programa, carga la página correspondiente desde la memoria secundaria a la memoria principal y actualiza la tabla de páginas. Una vez hecho esto, la instrucción que causó el fallo se reinicia, esta vez con la página ya disponible en la memoria principal. La Sección 12 presenta este tema en detalle.

Ejemplo

Supongamos que tenemos direcciones virtuales de 32 bits y páginas de $4\text{ KB} = 4096\text{ bytes} = 2^{12}\text{ bytes}$, por lo tanto la cantidad de bits para el desplazamiento dentro de la página es igual a 12. Además, supongamos que tenemos una memoria física de $1\text{ GB} = 2^{30}\text{ bytes}$. Por lo tanto, la cantidad de bits para especificar el número de marco de página es $30 - 12$; eso es 2^{18} páginas físicas. La memoria virtual para cada proceso cubre 4 GB (direcciones de memoria de 32 bits). Para especificar el número de página virtual la cantidad de bits es $32 - 12 = 20$, porque 12 es la cantidad de bits para el desplazamiento. El siguiente esquema ilustra este ejemplo:

Número de página virtual	V	Número de página física
0x7ffff	0	
0x7fffe	0	
0x7fffd	0	
0x7fffc	0	
0x7fffb	0	
0x7ffa	1	0x0001
⋮	⋮	⋮
0x00004	0	
0x00003	0	
0x00002	1	0x7ffe
0x00001	0	
0x00000	1	0x0000

Figura 9: Tabla de páginas.



Luego, la conversión (o traducción) entre el número de página lógica (o virtual) y el número de página física se realiza utilizando la **tabla de paginación** (o **tabla de páginas**). Supongamos la siguiente tabla de paginación (en realidad, solo mostramos un fragmento por cuestión de espacio):

	V	Número de página física
0x7fff9	1	0x3fe00
0x7fff8	1	0x3ff0c
0x7fff7	0	0x32056
0x7fff6	1	0x36000
0x7fff5	1	0x36020
0x7fff4	0	0x31c09
0x7fff3	0	0x3021f
0x7fff2	0	0x320a0
0x7fff1	1	0x31800
0x7fff0	0	0x3aa00

Veamos ahora cómo hacer una traducción de direcciones. Es decir, dada una dirección virtual determinar a cuál dirección física corresponde. Supongamos que partimos de la dirección virtual 0x7fff5a06. Si tenemos páginas de 4 kB, los 12 bits menos significativos corresponden al offset. En este caso, 0xa06. Luego, los restantes bits determinan el número de página virtual. En este caso, 0xffff5. Con este número de página virtual podemos entrar a la tabla de páginas (o tabla de paginación) anterior y obtener el número de página física. En este caso, 0x36020. Finalmente, la dirección física se obtiene concatenando el número de página física y el offset (o desplazamiento). Por lo tanto, la dirección física es 0x36020a06.

Observación

Como se ve en el ejemplo anterior, el número de bits para especificar una dirección física puede diferir del número de bits para especificar una dirección virtual. Por el contrario, el número de bits para especificar el desplazamiento de la dirección virtual y física es el mismo.

Observación

¿Por qué el tamaño de página es potencia de 2? Recordando que la paginación se implementa dividiendo una dirección en un número de página y un número de desplazamiento, y debido a que cada posición de bit representa una potencia de 2, da como resultado un tamaño de página que es una potencia de 2.

Por lo tanto, la traducción de direcciones (o conversión de direcciones) es un proceso por el cual una dirección virtual se convierte en una dirección que se utiliza para acceder a la memoria principal. Sin embargo, también es posible que una página virtual se encuentre ausente de la memoria principal y no se le haya asignado una dirección física, por lo que se encuentra en el disco. Se denomina **fallo de página** al suceso que ocurre cuando se quiere acceder a una página que no se encuentra en la memoria principal. El sistema operativo maneja la falla seleccionando una página “víctima”, intercambiando la página víctima si está sucia, intercambiando la página nueva y actualizando la tabla de páginas. Este mecanismo se denomina **swapping**.

Estas capacidades son proporcionadas por una combinación de software del sistema operativo, hardware de traducción de direcciones en la MMU (unidad de administración de memoria) y una estructura de datos almacenada en la memoria física conocida como **tabla de páginas** que asigna páginas virtuales a páginas físicas. El hardware de traducción de direcciones lee la tabla de páginas cada vez que convierte una dirección virtual en una dirección física. El sistema operativo es responsable de mantener el contenido de la tabla de páginas y transferir páginas de un lado a otro entre el disco y la DRAM.

8. Tamaño de página

El tamaño de la página es un parámetro que, en muchos casos, puede ser seleccionado por el sistema operativo entre varios tamaños compatibles con la arquitectura. En Linux, el tamaño de página por defecto es 4 KB en la mayoría de las arquitecturas (incluyendo x86 y ARM).

Determinar el tamaño de página óptimo implica equilibrar diversos factores en competencia, lo que significa que no existe una solución única que sea ideal en todos los casos. Por un lado, es común que un proceso no utilice completamente todas las páginas asignadas. En promedio, la mitad de la última página de un proceso quedará vacía, y ese espacio adicional se considera desperdiciado. Este tipo de desperdicio se conoce como **fragmentación interna**.

Se denomina fragmentación interna porque el espacio desperdiciado ocurre dentro de una página asignada. En particular, cuando el tamaño del proceso o del bloque de memoria solicitado no es un múltiplo exacto del tamaño de página, la última página asignada queda parcialmente sin utilizar. Ese espacio sobrante no puede aprovecharse para otros procesos, ya que forma parte de una página ya asignada, y por lo tanto constituye fragmentación interna.

Si hay n procesos y el tamaño de página es de p bytes, cada proceso puede desperdiciar hasta $p - 1$ bytes en su última página asignada, ya que el tamaño del proceso no suele ser un múltiplo exacto del tamaño de página. Bajo la suposición habitual de que el tamaño del desperdicio en la última página de cada proceso se distribuye de manera uniforme entre 0 y p bytes, el valor esperado del espacio no utilizado por proceso es $\frac{p}{2}$ bytes. En consecuencia, al considerar n procesos independientes, el desperdicio promedio total debido a la fragmentación interna se estima como $n \times \frac{p}{2}$ bytes.

Esto sugiere que utilizar páginas de menor tamaño podría reducir el espacio desperdiciado por fragmentación interna, ya que el máximo y el valor promedio del espacio no utilizado en la última página de cada proceso disminuyen al reducirse p . Sin embargo, un tamaño de página más pequeño implica que cada programa debe dividir su espacio de direcciones en un mayor número de páginas para cubrir la misma cantidad de memoria. Como consecuencia, las tablas de páginas deben contener más entradas, lo que incrementa su tamaño. Por lo tanto, existe un compromiso entre reducción de fragmentación interna y aumento del tamaño de las tablas de páginas. Una solución a este conflicto se aborda en la Sección 9.

9. Tablas de páginas multinivel

Como se mencionó en la Sección 7, un problema importante de la paginación es que las tablas de páginas pueden ocupar una gran cantidad de memoria física. Por ejemplo, la tabla de páginas para una memoria virtual de 32 bits con páginas de 4 KB necesita 2^{32} bytes/ 2^{12} bytes = 2^{20} entradas. Si cada entrada ocupa 4 bytes, la tabla de páginas ocupa $2^{20} \times 2^2$ bytes = 2^{22} bytes, es decir, 4 MB.

Para conservar memoria física, las tablas de páginas pueden dividirse en múltiples niveles. Este enfoque permite eliminar las regiones inválidas en lugar de mantenerlas completas en memoria. A esta técnica se la denomina **tabla de páginas multinivel** (o, de manera más coloquial, **paginación multinivel**), ya que transforma la tabla de páginas lineal en una estructura similar a un árbol. Es tan efectiva que muchos sistemas modernos la emplean (por ejemplo, x86).

La idea básica detrás de una tabla de páginas multinivel es la siguiente. En lugar de mantener una única tabla lineal con todas las entradas posibles, incluso las inválidas, la tabla se divide en varios niveles, donde cada nivel apunta al siguiente. Además, el número de página virtual de la dirección virtual se descompone en varios números de página, uno por cada nivel, y cada uno actúa como desplazamiento dentro de la tabla correspondiente. En la Fig. 10 se muestra un ejemplo simple con solo dos niveles de paginación. En la actualidad, las estructuras más complejas incluyen cuatro niveles⁵.

En una tabla de páginas de dos niveles, el número de página virtual se divide en dos partes: el **número de la tabla de páginas** y el **desplazamiento de la tabla de páginas**, como se muestra en la Fig. 10. El número de tabla de páginas indexa la tabla de páginas de primer nivel, que debe residir en la memoria física. El registro base de la tabla de páginas (PTBR) almacena la dirección base de la tabla de páginas que utiliza la MMU para comenzar la traducción de direcciones virtuales a físicas, y cambia de valor en cada cambio de contexto. La entrada de la tabla de páginas de primer nivel proporciona la dirección base de la tabla de páginas de

⁵Existen procesadores con cinco niveles que utilizan una extensión denominada *Intel 5-level paging*.

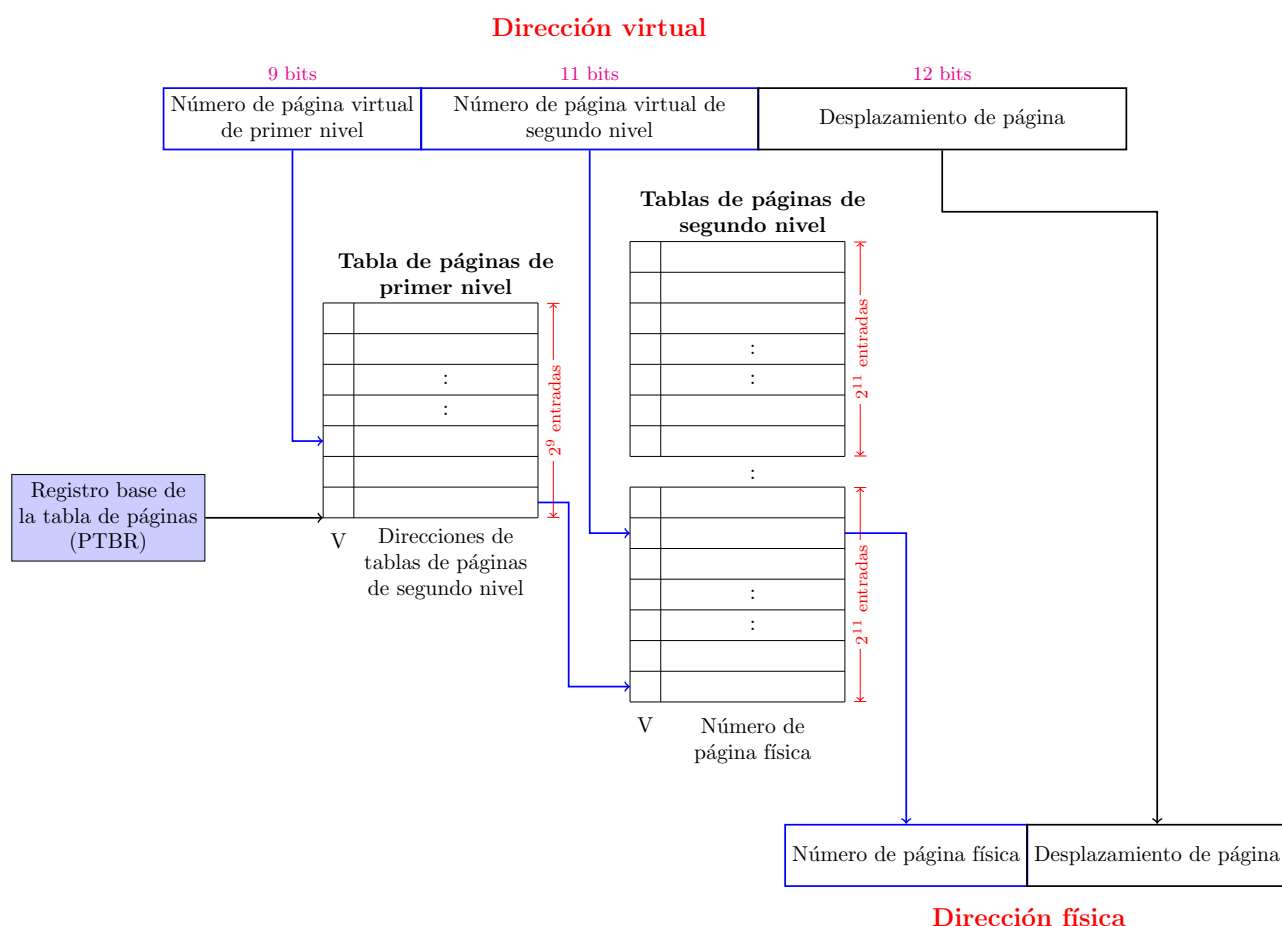


Figura 10: Ejemplo de paginación con tabla de páginas de dos niveles.

segundo nivel cuando $V = 1$, o indica que debe obtenerse del disco cuando $V = 0$. A su vez, el desplazamiento de la tabla de páginas indexa la tabla de páginas de segundo nivel. Los 12 bits restantes de la dirección virtual son el desplazamiento de página, suponiendo un tamaño de página de 4 KB (2^{12} bytes).

En el ejemplo de la Fig. 10, el número de página virtual de 20 bits se divide en 9 y 11 bits, para indicar el número de la tabla de páginas y el desplazamiento de la tabla de páginas, respectivamente. Así, la tabla de páginas de primer nivel tiene $2^9 = 512$ entradas. Cada una de estas 512 entradas apunta a una tabla de segundo nivel. A su vez, cada una de las tablas de segundo nivel tiene $2^{11} = 2048$ entradas. Si cada una de las entradas de la tabla de páginas de primer y segundo nivel es de 32 bits (4 bytes) y solo dos tablas de páginas de segundo nivel están presentes en la memoria física a la vez, la tabla de páginas jerárquica usa solo $512 \times 4 \text{ bytes} + 2 \times 2048 \times 4 \text{ bytes} = 18 \text{ KB}$ de memoria física. Por lo tanto, la tabla de páginas de dos niveles requiere una fracción de la memoria física necesaria para almacenar la tabla de páginas completa (4 MB). Sin embargo, el inconveniente de una tabla de páginas de dos niveles es que agrega otro acceso a la memoria para la traducción cuando “falla” la TLB.

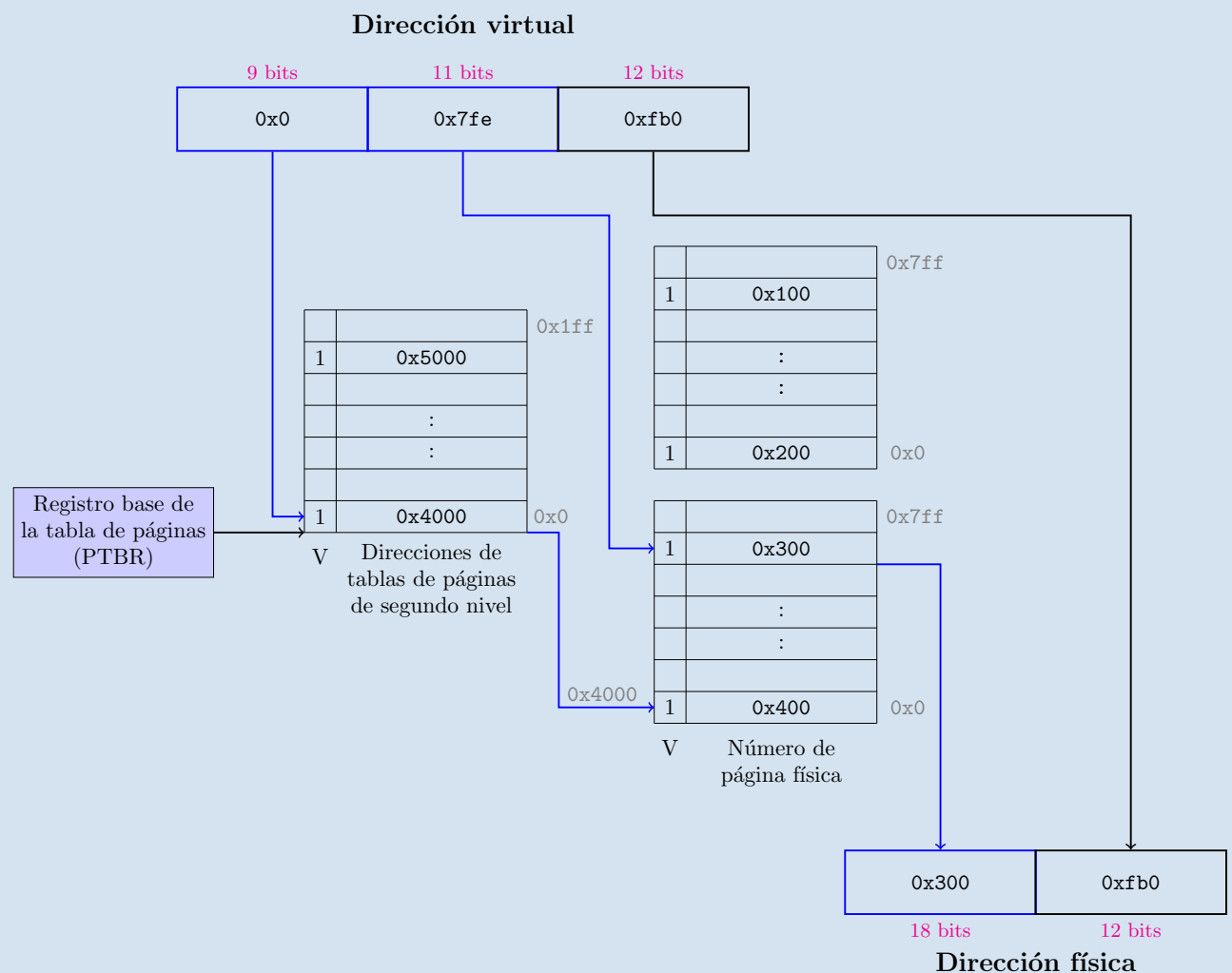
Ejemplo

La siguiente figura muestra, a modo de ejemplo, los posibles contenidos de una tabla de páginas de dos niveles, como la presentada en la Fig. 10. A partir de esta información, veremos cómo se realiza la traducción de la dirección virtual `0x7fxfb0`.

Como ya hemos visto, solamente el número de página virtual requiere traducción. Los nueve bits más significativos de la dirección virtual, `0x0`, dan el número de la tabla de páginas. Es

decir, el índice de la tabla de páginas de primer nivel. La tabla de páginas de primer nivel en la entrada **0x0** indica que la tabla de páginas de segundo nivel reside en la memoria (porque $V=1$) y su dirección física es **0x4000**. Los siguientes once bits de la dirección virtual, **0x7fe**, son el desplazamiento de la tabla de páginas, lo que proporciona el índice en la tabla de páginas de segundo nivel. La entrada **0x7fe** en la tabla de páginas de segundo nivel indica que la página virtual reside en la memoria física ($V=1$) y que el número de página física es **0x300**. El número de página física se concatena con el desplazamiento de página para formar la dirección física, la cual resulta **0x300fb0**.

Si la memoria física tiene un tamaño de 1 GB, entonces las direcciones físicas requieren 30 bits, ya que $1\text{ GB} = 2^{30}\text{ bytes}$. En general, las direcciones físicas utilizan menos bits que las direcciones virtuales, porque la memoria física suele ser considerablemente menor que el espacio de direcciones virtuales disponible.



10. Translation Lookaside Buffer (TLB)

Dado que cada operación de carga o almacenamiento puede implicar accesos adicionales a memoria para realizar la traducción de direcciones, el impacto en el rendimiento sería elevado si cada referencia requiriera recorrer la tabla de páginas. Sin embargo, los programas presentan **localidad temporal**, es decir, tienden a reutilizar en el corto plazo las mismas direcciones (o páginas) recientemente accedidas, y **localidad espacial**, lo que implica que direcciones cercanas

entre sí (dentro de una misma página o en páginas contiguas) tienen alta probabilidad de ser accedidas en forma consecutiva. Como resultado, muchas referencias de memoria corresponden a páginas cuyas traducciones ya fueron utilizadas recientemente. Este comportamiento permite que la Unidad de Manejo de Memoria (MMU) almacene en la TLB (Translation Lookaside Buffer) entradas recientes de la tabla de páginas y reutilice dichas traducciones, evitando accesos repetidos a la memoria y reduciendo significativamente el costo de la traducción de direcciones.

En general, el procesador almacena las últimas entradas de la tabla de páginas en una pequeña tabla llamada **Translation Lookaside Buffer (TLB)**, que está ubicada dentro de la memoria caché. El procesador primero intenta encontrar la traducción en la TLB antes de acceder a la tabla de páginas en la memoria física, como se muestra en la Fig. 11. Si la traducción no se encuentra en la TLB (*TLB miss*), el procesador debe consultar la tabla de páginas en memoria para obtenerla y, típicamente, incorporar luego esta entrada a la TLB para futuros accesos. En la práctica, la gran mayoría de los accesos se realizan utilizando la TLB, lo que evita las costosas lecturas de la tabla de páginas desde la memoria física.

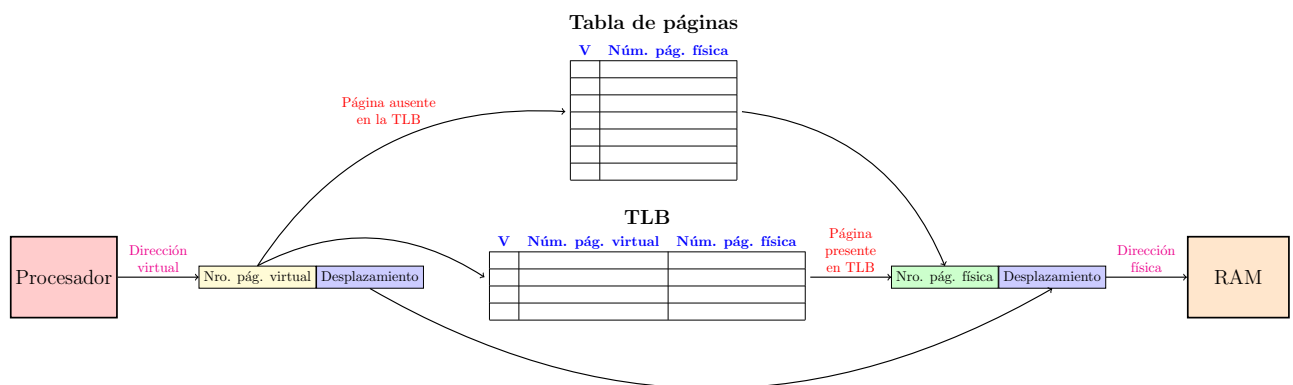


Figura 11: Esquema de traducción con TLB.

La TLB se organiza como una memoria caché totalmente asociativa y generalmente contiene entre 16 y 512 entradas. Cada entrada de la TLB incluye un número de página virtual, su correspondiente número de página física y algunas banderas de estado. Se accede a la TLB utilizando el número de página virtual. Si la TLB tiene un “acierto”, devuelve el número de página física correspondiente; de lo contrario, el procesador debe leer la tabla de páginas en la memoria física. La TLB está diseñada para ser lo suficientemente pequeña como para permitir su acceso en menos de un ciclo de reloj. A pesar de su tamaño compacto, las TLB suelen tener una tasa de aciertos superior al 99 %. Gracias a la TLB, se reduce el número de accesos a la memoria necesarios para la mayoría de las instrucciones de carga o almacenamiento de dos a uno (e incluso más en sistemas con paginación multinivel).

Ejemplo

Continuando con el ejemplo anterior, supongamos ahora que además de la tabla de páginas tenemos una TLB con solo 4 entradas como se muestra a continuación:

V	Número de página virtual	Número de página físico
0	0xefff0	0x0aa00
1	0xffff5	0x36020
1	0x0fff4	0x4800a
0	0xaaff3	0x7a001

Supongamos que queremos traducir la dirección virtual 0xffff5a06 como en el ejemplo anterior. La TLB recibe el número de página virtual de la dirección entrante, 0xffff5, y lo compara con el número de página virtual de cada entrada. La entrada 1 coincide y es válida, por lo que la traducción se puede realizar y no es necesario acceder a memoria. La dirección física traducida es el número de página física de la entrada coincidente, 0x36020, concatenado con el desplazamiento de página de la dirección virtual. Como siempre, el desplazamiento de página no requiere traducción. Al contrario, la solicitud de la dirección virtual 0xffff8 no se encuentra en la TLB. Por lo tanto, la solicitud se envía a la tabla de páginas para su traducción y además luego se carga la traducción en la TLB.

11. Protección de memoria

Hasta ahora, nos hemos centrado en el uso de la memoria virtual para proporcionar una memoria grande, rápida y económica. Una razón igualmente importante para usar la memoria virtual es brindar protección entre programas que se ejecutan simultáneamente. Las computadoras modernas normalmente ejecutan varios programas o procesos al mismo tiempo. Todos los programas están simultáneamente presentes en la memoria física. En un sistema informático bien diseñado, los programas deben estar protegidos entre sí para que ningún programa pueda interferir en otro programa. Específicamente, ningún programa debería poder acceder a la memoria de otro programa sin permiso. Esto se llama **protección de la memoria**.

Los sistemas de memoria virtual brindan protección a la memoria al otorgar a cada programa su propio **espacio de direcciones virtuales**. Cada programa puede usar tanta memoria como quiera en ese espacio de direcciones virtuales, pero puede que solo una parte del espacio de direcciones virtuales esté en la memoria física en un momento dado. Cada programa puede utilizar todo su espacio de direcciones virtuales sin tener que preocuparse por la ubicación física de otros programas. Sin embargo, un programa puede acceder solo a aquellas páginas físicas que están asignadas en su tabla de páginas. De esta forma, un programa no puede acceder accidental o maliciosamente a las páginas físicas de otro programa, porque no están mapeadas en su tabla de páginas. En algunos casos, varios programas acceden a instrucciones o datos comunes. El sistema operativo agrega bits de control a cada entrada de la tabla de páginas para determinar qué programas, si los hay, pueden escribir en las páginas físicas compartidas.

12. Políticas de reemplazo

La **política de reemplazo** define el criterio que utiliza el sistema operativo para decidir qué página debe ser retirada de la memoria principal cuando se produce un fallo de página y no hay marcos libres disponibles. Dado que la memoria física es un recurso limitado, la elección de la página a reemplazar tiene un impacto directo en el rendimiento del sistema, en particular sobre la frecuencia de fallos de página. Por este motivo, las políticas de reemplazo constituyen un componente central de la gestión de memoria en los sistemas operativos.

En este contexto, el diseño de una política de reemplazo busca aproximarse al comportamiento óptimo, es decir, minimizar la cantidad de fallos de página a lo largo del tiempo. Para ello, las distintas estrategias se apoyan en información histórica sobre los accesos a memoria o en heurísticas que intentan capturar la localidad temporal y espacial de los programas. Sin embargo, existe un compromiso entre la calidad de la decisión y el costo de implementarla, ya que las políticas más precisas suelen requerir mayor sobrecarga en términos de hardware o de gestión por parte del sistema operativo.

Desde el punto de vista operativo, una política de reemplazo debe definir qué información se mantiene para cada página y en qué momento se actualiza. Algunas políticas registran el instante del último acceso, otras contabilizan la frecuencia de uso, mientras que las más simples se basan en decisiones aproximadas o aleatorias. Estas diferencias dan lugar a algoritmos clásicos como FIFO (*First-In, First-Out*), LRU (*Least Recently Used*) y sus diversas aproximaciones, cuya implementación concreta depende tanto del soporte de hardware disponible como de los objetivos de rendimiento del sistema operativo.

El algoritmo FIFO reemplaza la página que lleva más tiempo en memoria, independientemente de si fue utilizada recientemente o no. Las páginas se organizan conceptualmente en una cola: cuando una página ingresa a memoria se coloca al final, y ante la necesidad de reemplazo se elimina la que se encuentra al frente. La principal ventaja es su implementación simple y de bajo costo. La desventaja es que no tiene en cuenta el patrón real de uso de las páginas, por lo que puede reemplazar páginas que aún se encuentran activas. Esto puede dar lugar a comportamientos anómalos, como la anomalía de Belady, en la cual aumentar la cantidad de marcos disponibles incrementa el número de fallos de página.

El algoritmo LRU reemplaza la página que no ha sido utilizada durante más tiempo, basándose en la idea de localidad temporal: una página usada recientemente tiene mayor probabilidad de volver a usarse en el corto plazo. Para ello, el sistema debe llevar un registro del orden o del instante del último acceso a cada página. La principal ventaja es que suele aproximar mejor el comportamiento óptimo y reducir la cantidad de fallos de página. La desventaja es que su implementación exacta resulta costosa, ya que requiere soporte de hardware o el uso de estructuras de datos complejas para actualizar la información en cada acceso.

Dado que la implementación exacta de LRU resulta costosa computacionalmente, en la práctica se emplean aproximaciones que intentan capturar el uso reciente de las páginas con menor sobrecarga. Estas aproximaciones suelen apoyarse en bits de referencia provistos por el hardware, que se activan cuando una página es accedida. El sistema operativo puede inspeccionar y reinicializar periódicamente estos bits para estimar qué páginas han sido utilizadas recientemente y cuáles no, tomando así decisiones de reemplazo que, sin ser estrictamente LRU, se aproximan razonablemente a su comportamiento.

Un procedimiento típico para aproximar LRU se basa en el uso sistemático de los bits de referencia (R), también denominado bit de uso (U), y de modificación (M), también conocido como bit de “suciedad” (*dirty bit*), asociados a cada página. En primer lugar, el hardware pone en 1 el bit R cada vez que la página es accedida y el bit M cuando la página es escrita. A intervalos regulares, el sistema operativo inicia un recorrido secuencial de las páginas residentes en memoria y examina estos bits. Durante este recorrido, el bit R se reinicia a 0 para todas las páginas, de modo que en el siguiente intervalo pueda detectarse cuáles han sido referenciadas nuevamente. Cuando se produce un fallo de página y es necesario seleccionar una “víctima”, el sistema operativo clasifica las páginas según las combinaciones de (R, M) —por ejemplo, $(0, 0)$, $(0, 1)$, $(1, 0)$ y $(1, 1)$ — y elige preferentemente una página con $R = 0$, ya que no ha sido utilizada recientemente, y con $M = 0$, ya que no requiere ser escrita a memoria secundaria antes de su reemplazo. Este procedimiento, aunque no reproduce exactamente LRU, permite aproximar su comportamiento de manera eficiente y con bajo costo de implementación.

13. Caso de estudio: Intel Core i7

Un caso de estudio real interesante para estudiar los mecanismos de memoria virtual es el procesador Intel Core i7 ejecutando Linux. Aunque la arquitectura x86-64 permite espacios completos de direcciones físicas y virtuales de 64 bits, las implementaciones actuales de Intel Core i7 (y las del futuro previsible) admiten un espacio de direcciones virtuales de 48 bits (256 TB) y un espacio de direcciones físicas de 52 bits (4 PB), junto con un modo de

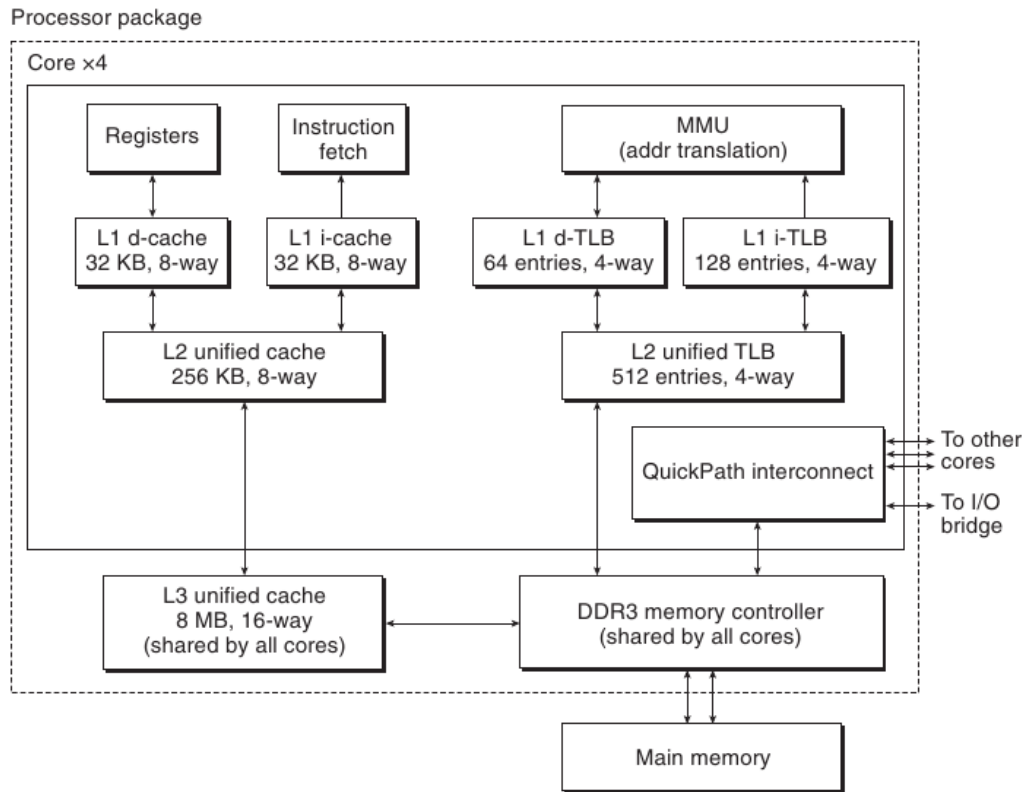


Figura 12: Sistema de memoria del procesador Intel Core i7 (fuente: [2]).

compatibilidad que admite espacios de direcciones físicas y virtuales de 32 bits (4 GB).

La Fig. 12 muestra los aspectos más destacados del sistema de memoria del procesador Intel Core i7. El chip incluye cuatro núcleos⁶, una gran caché L3 compartida por todos los núcleos y un controlador de memoria DDR3, entre otros módulos. Cada núcleo contiene una jerarquía de TLB y una jerarquía de cachés de datos e instrucciones. La TLB de cada núcleo es asociativa de 4 vías. L1 y L2 son asociativas de 8 vías, y L3 es asociativa de 16 vías. El tamaño de la página se puede configurar en el momento del inicio como 4 KB o 4 MB. Linux usa páginas de 4 KB.

El procesador Intel Core i7 utiliza una jerarquía de tablas de páginas de cuatro niveles. Cada proceso tiene su propia jerarquía de tablas de páginas privadas. Cuando se ejecuta un proceso de Linux, las tablas de páginas asociadas con las páginas asignadas residen todas en la memoria, aunque el procesador Intel Core i7 permite que estas tablas de páginas se intercambien y se desconecten. El registro de control CR3 contiene la dirección física del comienzo de la tabla de páginas de nivel 1 (L1). El valor de CR3 forma parte del contexto de cada proceso, ya que identifica la base de la estructura de traducción de direcciones asociada a ese proceso. Durante un cambio de contexto, el sistema operativo restaura el contenido de CR3 junto con el resto de los registros de estado del proceso entrante, de modo que el procesador comience a traducir direcciones virtuales utilizando las tablas correspondientes a ese proceso. Este mecanismo garantiza el aislamiento entre espacios de direcciones y permite que cada proceso tenga su propia vista de la memoria⁷.

Las direcciones virtuales son de 48 bits: 36 bits para el número de página virtual (9 bits para cada nivel) y 12 bits para el offset, mientras que el ancho de las direcciones físicas no es fijo, sino que depende de la implementación particular del procesador dentro de la familia

⁶Intel Core i7 es una familia de procesadores con diferentes modelos. El modelo mostrado en la Fig. 12 tiene 4 núcleos pero hay modelos con mayor cantidad de núcleos.

⁷Este tema se verá en detalle en la asignatura Sistemas Operativos II.

Referencias

- [1] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, first edition, 2018.
- [2] Randal E Bryant, O'Hallaron David Richard, and O'Hallaron David Richard. *Computer systems: a programmer's perspective*. Pearson Education Limited, third edition, 2015.
- [3] Ulrich Drepper. What every programmer should know about memory. *Red Hat, Inc*, 11(2007):2007, 2007.
- [4] David Money Harris and Sarah L. Harris. *Digital Design and Computer Architecture, Second Edition*. Morgan Kaufmann, 2012.
- [5] John L Hennessy and David A Patterson. *Computer architecture: a quantitative approach*. Elsevier, 2011.
- [6] David A Patterson, John L Hennessy, and Peter J Ashenden. *Computer Organization and Design: The Hardware/Software Interface*. Elsevier Science Limited, fifth edition, 2014.
- [7] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating system concepts*. 10th edition, 2018.
- [8] William Stallings and Goutam Kumar Paul. *Operating systems: internals and design principles*, volume 9. Pearson New York, 2012.
- [9] Andrew S Tanenbaum. *Organización de computadoras: un enfoque estructurado*. Pearson educación, 2000.
- [10] Gunnar Wolf, Esteban Ruiz, Federico Bergero, and Erwin Meza. *Fundamentos de sistemas operativos*. Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas: Facultad de Ingeniería, 2015.
- [11] Igor Zhirkov. *Low-Level Programming: C, Assembly, and Program Execution on Intel 64 Architecture*. Apress, 2017.